

-- DBMS SIMULATION: employee attendance, shift and leave management
-- Dialect: SQLite. Run in a NEW, EMPTY database.
-- Foreign keys, constraints, triggers, sample data and SELECT reports.

PRAGMA foreign_keys = ON;

```
CREATE TABLE IF NOT EXISTS departments (  
    department_id INTEGER PRIMARY KEY,  
    department_name TEXT NOT NULL UNIQUE  
        CHECK (length(trim(department_name)) BETWEEN 1 AND 80)  
);
```

```
CREATE TABLE IF NOT EXISTS employees (  
    employee_id INTEGER PRIMARY KEY,  
    employee_name TEXT NOT NULL  
        CHECK (length(trim(employee_name)) BETWEEN 1 AND 100),  
    email TEXT NOT NULL UNIQUE COLLATE NOCASE,  
    department_id INTEGER NOT NULL,  
    FOREIGN KEY (department_id) REFERENCES departments(department_id)  
);
```

-- Times are minutes after midnight: 09:00 = 540, 17:00 = 1020.
-- If end_minute < start_minute, the shift ends on the following day.

```
CREATE TABLE IF NOT EXISTS shifts (  
    shift_id INTEGER PRIMARY KEY,  
    shift_name TEXT NOT NULL UNIQUE  
        CHECK (length(trim(shift_name)) BETWEEN 1 AND 80),  
    start_minute INTEGER NOT NULL CHECK (start_minute BETWEEN 0 AND 1439),  
    end_minute INTEGER NOT NULL CHECK (end_minute BETWEEN 0 AND 1439),  
    grace_minutes INTEGER NOT NULL DEFAULT 10  
        CHECK (grace_minutes BETWEEN 0 AND 60),  
    CHECK (start_minute != end_minute)  
);
```

```
CREATE TABLE IF NOT EXISTS shift_assignments (  
    assignment_id INTEGER PRIMARY KEY,  
    employee_id INTEGER NOT NULL,  
    work_date TEXT NOT NULL CHECK (  
        length(work_date) = 10 AND date(work_date, '+0 days') IS NOT NULL  
        AND date(work_date, '+0 days') = work_date  
    ),  
    shift_id INTEGER NOT NULL,  
    UNIQUE (employee_id, work_date),  
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id),  
    FOREIGN KEY (shift_id) REFERENCES shifts(shift_id)  
);
```

```
CREATE TABLE IF NOT EXISTS attendance (  
    attendance_id INTEGER PRIMARY KEY,  
    assignment_id INTEGER NOT NULL UNIQUE,  
    check_in TEXT NOT NULL CHECK (  
        length(check_in) = 19 AND datetime(check_in) IS NOT NULL  
        AND datetime(check_in, '+0 days') = check_in  
    ),  
    check_out TEXT CHECK (  
        check_out IS NULL OR (  
            length(check_out) = 19 AND datetime(check_out) IS NOT NULL  
            AND datetime(check_out, '+0 days') = check_out  
            AND check_out > check_in  
        )  
    )
```

```

),
FOREIGN KEY (assignment_id) REFERENCES shift_assignments(assignment_id)
);

CREATE TABLE IF NOT EXISTS leave_requests (
    leave_id INTEGER PRIMARY KEY,
    employee_id INTEGER NOT NULL,
    leave_type TEXT NOT NULL CHECK (leave_type IN ('Casual', 'Sick', 'Earned')),
    start_date TEXT NOT NULL CHECK (
        length(start_date) = 10 AND date(start_date, '+0 days') IS NOT NULL
        AND date(start_date, '+0 days') = start_date
    ),
    end_date TEXT NOT NULL CHECK (
        length(end_date) = 10 AND date(end_date, '+0 days') IS NOT NULL
        AND date(end_date, '+0 days') = end_date AND end_date >= start_date
    ),
    reason TEXT NOT NULL CHECK (length(trim(reason)) BETWEEN 1 AND 250),
    status TEXT NOT NULL DEFAULT 'Pending'
    CHECK (status IN ('Pending', 'Approved', 'Rejected')),
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id)
);

CREATE INDEX IF NOT EXISTS idx_assignments_date ON shift_assignments(work_date);
CREATE INDEX IF NOT EXISTS idx_leave_employee_dates
    ON leave_requests(employee_id, start_date, end_date, status);

-- Derive full timestamps once so overnight shifts work in every report.
CREATE VIEW IF NOT EXISTS v_schedule AS
SELECT a.assignment_id, a.employee_id, a.work_date, a.shift_id,
    s.shift_name, s.grace_minutes,
    datetime(a.work_date, printf('+%d minutes', s.start_minute)) AS shift_start,
    datetime(a.work_date,
        CASE WHEN s.end_minute < s.start_minute THEN '+1 day' ELSE '+0 days' END,
        printf('+%d minutes', s.end_minute)) AS shift_end
FROM shift_assignments a JOIN shifts s ON s.shift_id = a.shift_id;

-- Check conflicts inside the database as well as in the application.
CREATE TRIGGER IF NOT EXISTS prevent_overlapping_shift
AFTER INSERT ON shift_assignments
WHEN EXISTS (
    SELECT 1 FROM v_schedule n JOIN v_schedule old
    ON old.employee_id = n.employee_id AND old.assignment_id != n.assignment_id
    AND n.shift_start < old.shift_end AND n.shift_end > old.shift_start
    WHERE n.assignment_id = NEW.assignment_id
)
BEGIN SELECT RAISE(ABORT, 'This shift overlaps an existing assignment.');
```

```

END;

-- Assignments and used shift definitions are immutable in this teaching model.
CREATE TRIGGER IF NOT EXISTS prevent_assignment_update
BEFORE UPDATE ON shift_assignments
BEGIN SELECT RAISE(ABORT, 'Assignments are fixed; create a new schedule entry.');
```

```

END;

CREATE TRIGGER IF NOT EXISTS protect_used_shift
BEFORE UPDATE ON shifts
WHEN EXISTS (SELECT 1 FROM shift_assignments WHERE shift_id = OLD.shift_id)
BEGIN SELECT RAISE(ABORT, 'Create a new shift definition to preserve history.');
```

```

END;

CREATE TRIGGER IF NOT EXISTS leave_overlap_insert
BEFORE INSERT ON leave_requests
WHEN NEW.status != 'Rejected' AND EXISTS (
```

```

SELECT 1 FROM leave_requests l
WHERE l.employee_id = NEW.employee_id AND l.status != 'Rejected'
AND NEW.start_date <= l.end_date AND NEW.end_date >= l.start_date
)
BEGIN SELECT RAISE(ABORT, 'An active leave request already covers these dates.');
```

END;

```

CREATE TRIGGER IF NOT EXISTS leave_overlap_update
BEFORE UPDATE ON leave_requests
WHEN NEW.status != 'Rejected' AND EXISTS (
  SELECT 1 FROM leave_requests l
  WHERE l.employee_id = NEW.employee_id AND l.leave_id != OLD.leave_id
  AND l.status != 'Rejected'
  AND NEW.start_date <= l.end_date AND NEW.end_date >= l.start_date
)
BEGIN SELECT RAISE(ABORT, 'An active leave request already covers these dates.');
```

END;

```

CREATE TRIGGER IF NOT EXISTS approved_leave_insert
BEFORE INSERT ON leave_requests
WHEN NEW.status = 'Approved' AND EXISTS (
  SELECT 1 FROM attendance t JOIN shift_assignments a
  ON t.assignment_id = a.assignment_id
  WHERE a.employee_id = NEW.employee_id
  AND a.work_date BETWEEN NEW.start_date AND NEW.end_date
)
BEGIN SELECT RAISE(ABORT, 'Attendance already exists on a requested leave date.');
```

END;

```

CREATE TRIGGER IF NOT EXISTS approved_leave_update
BEFORE UPDATE ON leave_requests
WHEN NEW.status = 'Approved' AND EXISTS (
  SELECT 1 FROM attendance t JOIN shift_assignments a
  ON t.assignment_id = a.assignment_id
  WHERE a.employee_id = NEW.employee_id
  AND a.work_date BETWEEN NEW.start_date AND NEW.end_date
)
BEGIN SELECT RAISE(ABORT, 'Attendance already exists on a requested leave date.');
```

END;

```

CREATE TRIGGER IF NOT EXISTS attendance_leave_insert
BEFORE INSERT ON attendance
WHEN EXISTS (
  SELECT 1 FROM shift_assignments a JOIN leave_requests l
  ON l.employee_id = a.employee_id
  WHERE a.assignment_id = NEW.assignment_id AND l.status = 'Approved'
  AND a.work_date BETWEEN l.start_date AND l.end_date
)
BEGIN SELECT RAISE(ABORT, 'Cannot mark attendance during approved leave.');
```

END;

```

CREATE TRIGGER IF NOT EXISTS attendance_leave_update
BEFORE UPDATE ON attendance
WHEN EXISTS (
  SELECT 1 FROM shift_assignments a JOIN leave_requests l
  ON l.employee_id = a.employee_id
  WHERE a.assignment_id = NEW.assignment_id AND l.status = 'Approved'
  AND a.work_date BETWEEN l.start_date AND l.end_date
)
BEGIN SELECT RAISE(ABORT, 'Cannot mark attendance during approved leave.');
```

END;

```

-- Check-in is allowed from 60 minutes before the shift until its end.
-- Checkout is allowed up to 24 hours after the scheduled shift start.
CREATE TRIGGER IF NOT EXISTS attendance_time_insert
BEFORE INSERT ON attendance
```

```

WHEN EXISTS (
    SELECT 1 FROM v_schedule s WHERE s.assignment_id = NEW.assignment_id
    AND (NEW.check_in < datetime(s.shift_start, '-60 minutes')
        OR NEW.check_in >= s.shift_end
        OR NEW.check_out > datetime(s.shift_start, '+24 hours'))
)
BEGIN SELECT RAISE(ABORT, 'Attendance time is outside the allowed shift window.');
```

```

END;

CREATE TRIGGER IF NOT EXISTS attendance_time_update
BEFORE UPDATE ON attendance
WHEN EXISTS (
    SELECT 1 FROM v_schedule s WHERE s.assignment_id = NEW.assignment_id
    AND (NEW.check_in < datetime(s.shift_start, '-60 minutes')
        OR NEW.check_in >= s.shift_end
        OR NEW.check_out > datetime(s.shift_start, '+24 hours'))
)
BEGIN SELECT RAISE(ABORT, 'Attendance time is outside the allowed shift window.');
```

```

END;

BEGIN TRANSACTION;
```

```

-- Sample data: departments
```

```

INSERT INTO departments VALUES (1, 'IT');
INSERT INTO departments VALUES (2, 'Human Resources');
INSERT INTO departments VALUES (3, 'Operations');
```

```

-- Sample data: employees
```

```

INSERT INTO employees VALUES (1, 'Ananya Rao', 'ananya@example.com', 1);
INSERT INTO employees VALUES (2, 'Ravi Kumar', 'ravi@example.com', 2);
INSERT INTO employees VALUES (3, 'Sara Ali', 'sara@example.com', 3);
```

```

-- Sample data: shifts
```

```

INSERT INTO shifts VALUES (1, 'Day', 540, 1020, 10);
INSERT INTO shifts VALUES (2, 'Evening', 840, 1320, 10);
INSERT INTO shifts VALUES (3, 'Night', 1320, 360, 10);
```

```

-- Sample data: shift_assignments
```

```

INSERT INTO shift_assignments VALUES (1, 1, '2026-10-05', 1);
INSERT INTO shift_assignments VALUES (2, 2, '2026-10-05', 1);
INSERT INTO shift_assignments VALUES (3, 3, '2026-10-05', 3);
INSERT INTO shift_assignments VALUES (4, 1, '2026-10-06', 1);
INSERT INTO shift_assignments VALUES (5, 2, '2026-10-06', 1);
INSERT INTO shift_assignments VALUES (6, 3, '2026-10-06', 3);
INSERT INTO shift_assignments VALUES (7, 1, '2026-10-07', 1);
INSERT INTO shift_assignments VALUES (8, 2, '2026-10-07', 1);
INSERT INTO shift_assignments VALUES (9, 3, '2026-10-07', 3);
```

```

-- Sample data: attendance
```

```

INSERT INTO attendance VALUES (1, 1, '2026-10-05 08:57:00', '2026-10-05 17:05:00');
INSERT INTO attendance VALUES (2, 2, '2026-10-05 09:18:00', '2026-10-05 17:00:00');
INSERT INTO attendance VALUES (3, 3, '2026-10-05 22:00:00', '2026-10-06 06:00:00');
INSERT INTO attendance VALUES (4, 6, '2026-10-06 22:05:00', NULL);
```

```

-- Sample data: leave_requests
```

```

INSERT INTO leave_requests VALUES (1, 1, 'Sick', '2026-10-06', '2026-10-06', 'Medical rest',
    'Approved');
INSERT INTO leave_requests VALUES (2, 2, 'Casual', '2026-10-06', '2026-10-06', 'Personal
work', 'Rejected');
INSERT INTO leave_requests VALUES (3, 1, 'Casual', '2026-10-07', '2026-10-07', 'Family
commitment', 'Pending');
```

COMMIT;

-- Report 1: all employees and departments

```
SELECT e.employee_id, e.employee_name, d.department_name
FROM employees e JOIN departments d ON d.department_id = e.department_id;
```

-- Report 2: leave requests and decisions

```
SELECT e.employee_name, l.leave_type, l.start_date, l.end_date, l.status
FROM leave_requests l JOIN employees e ON e.employee_id = l.employee_id;
```

-- Daily report for 2026-10-05, evaluated at 2026-10-08 08:00:00

```
SELECT s.work_date, e.employee_id, e.employee_name, d.department_name,
       s.shift_name, s.shift_start, s.shift_end,
       CASE WHEN t.check_in <= '2026-10-08 08:00:00' THEN t.check_in END AS check_in,
       CASE WHEN t.check_out <= '2026-10-08 08:00:00' THEN t.check_out END AS check_out,
       CASE
         WHEN EXISTS (
           SELECT 1 FROM leave_requests l WHERE l.employee_id = e.employee_id
           AND l.status = 'Approved'
           AND s.work_date BETWEEN l.start_date AND l.end_date
         ) THEN 'On Leave'
         WHEN t.check_in <= '2026-10-08 08:00:00' AND (t.check_out IS NULL OR t.check_out >
'2026-10-08 08:00:00')
           THEN CASE WHEN s.shift_end <= '2026-10-08 08:00:00' THEN 'Missing Checkout'
                     ELSE 'In Progress' END
         WHEN t.check_in <= '2026-10-08 08:00:00' THEN
           CASE WHEN t.check_in > datetime(s.shift_start,
             printf('+%d minutes', s.grace_minutes))
             THEN 'Late' ELSE 'Present' END
         WHEN s.shift_end <= '2026-10-08 08:00:00' THEN 'Absent'
         WHEN s.shift_start <= '2026-10-08 08:00:00' THEN 'Awaiting Check-in'
         ELSE 'Scheduled'
       END AS status,
       CASE WHEN t.check_in <= '2026-10-08 08:00:00'
         AND t.check_in > datetime(s.shift_start, printf('+%d minutes', s.grace_minutes))
         THEN CAST((strftime('%s', t.check_in) - strftime('%s', s.shift_start)) / 60 AS INTEGER)
         ELSE 0 END AS late_minutes,
       CASE WHEN t.check_out <= '2026-10-08 08:00:00' THEN ROUND(
         (strftime('%s', t.check_out) - strftime('%s', t.check_in)) / 3600.0, 2)
         ELSE 0 END AS worked_hours
FROM v_schedule s
JOIN employees e ON e.employee_id = s.employee_id
JOIN departments d ON d.department_id = e.department_id
LEFT JOIN attendance t ON t.assignment_id = s.assignment_id
WHERE s.work_date BETWEEN '2026-10-05' AND '2026-10-05'
ORDER BY e.employee_id;
```

-- Daily report for 2026-10-06, evaluated at 2026-10-08 08:00:00

```
SELECT s.work_date, e.employee_id, e.employee_name, d.department_name,
       s.shift_name, s.shift_start, s.shift_end,
       CASE WHEN t.check_in <= '2026-10-08 08:00:00' THEN t.check_in END AS check_in,
       CASE WHEN t.check_out <= '2026-10-08 08:00:00' THEN t.check_out END AS check_out,
       CASE
         WHEN EXISTS (
           SELECT 1 FROM leave_requests l WHERE l.employee_id = e.employee_id
           AND l.status = 'Approved'
           AND s.work_date BETWEEN l.start_date AND l.end_date
         ) THEN 'On Leave'
         WHEN t.check_in <= '2026-10-08 08:00:00' AND (t.check_out IS NULL OR t.check_out >
'2026-10-08 08:00:00')
```

```

        THEN CASE WHEN s.shift_end <= '2026-10-08 08:00:00' THEN 'Missing Checkout'
        ELSE 'In Progress' END
    WHEN t.check_in <= '2026-10-08 08:00:00' THEN
        CASE WHEN t.check_in > datetime(s.shift_start,
            printf('+%d minutes', s.grace_minutes))
        THEN 'Late' ELSE 'Present' END
    WHEN s.shift_end <= '2026-10-08 08:00:00' THEN 'Absent'
    WHEN s.shift_start <= '2026-10-08 08:00:00' THEN 'Awaiting Check-in'
    ELSE 'Scheduled'
END AS status,
CASE WHEN t.check_in <= '2026-10-08 08:00:00'
AND t.check_in > datetime(s.shift_start, printf('+%d minutes', s.grace_minutes))
THEN CAST((strftime('%s', t.check_in) - strftime('%s', s.shift_start)) / 60 AS INTEGER)
ELSE 0 END AS late_minutes,
CASE WHEN t.check_out <= '2026-10-08 08:00:00' THEN ROUND(
    (strftime('%s', t.check_out) - strftime('%s', t.check_in)) / 3600.0, 2)
ELSE 0 END AS worked_hours
FROM v_schedule s
JOIN employees e ON e.employee_id = s.employee_id
JOIN departments d ON d.department_id = e.department_id
LEFT JOIN attendance t ON t.assignment_id = s.assignment_id
WHERE s.work_date BETWEEN '2026-10-06' AND '2026-10-06'
ORDER BY e.employee_id;

```